



DEPARTMENT OF
COMPUTER SCIENCE



COMPUTATIONAL APPROACHES TO PROBLEM SOLVING I

DR JEREMIAH O. BANDELE

PhD Electrical/Electronic Engineering
University of Nottingham



LEARNING OBJECTIVES

At the end of this lesson, you should be able to:

- Explain what computational approaches are.
- Identify the factors that determine if computational approaches can be used for a problem.
- Discuss types of computational approaches.



COMPUTATIONAL APPROACHES TO PROBLEM SOLVING

Computational approaches express solutions to problems in a form that can be run on a computer using advances in computing such as algorithms, models and simulations.

Before a problem can be solved using computational approaches, some factors need to be considered:

- **The problem needs to be clearly defined:** Identify the current situation, end goal, how to reach the end goal and possible obstacles.



COMPUTATIONAL APPROACHES TO PROBLEM SOLVING

- **The problem needs to be computable:** Determine if the required calculations are possible, considering processing times and capacity.
- **The data requirements of the problem need to be examined:** Identify the required data types and storage capacity.

After all this factors are considered, the decision of if computational methods can be used to solve the problem can be made.



COMPUTATIONAL APPROACHES TO PROBLEM SOLVING

Examples of computational methods that can be used to help solve complex problems includes:

- Problem recognition
- Backtracking
- Data mining
- Performance testing
- Pipelining
- Visualisation



PROBLEM RECOGNITION

To ensure that the proposed solution will deal with the root cause of a given problem, some factors need to be considered:

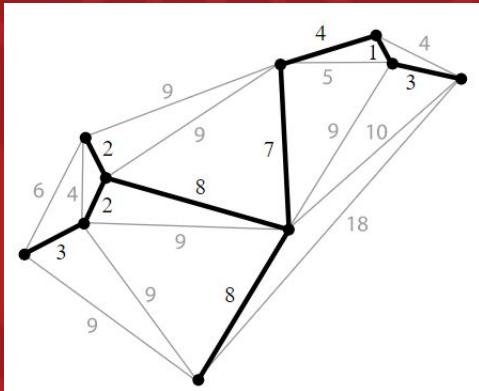
- The nature and parameters of the problem should be identified.
- The problem requirement or criteria for success should be specified.
- Determine if the problem can be grouped as a specific type of problem.



PROBLEM RECOGNITION

The possibility of reducing the problem to a more general case can also be explored.

For example, a problem to produce an algorithm that optimises the power cable infrastructure of a building can be reduced to determining the 'minimum spanning tree' from a given graph





BACKTRACKING

This is a problem solving method that involves examining more than one possible solution.

Most times, backtracking algorithms use recursion in order to move from one problem state to another.



BACKTRACKING

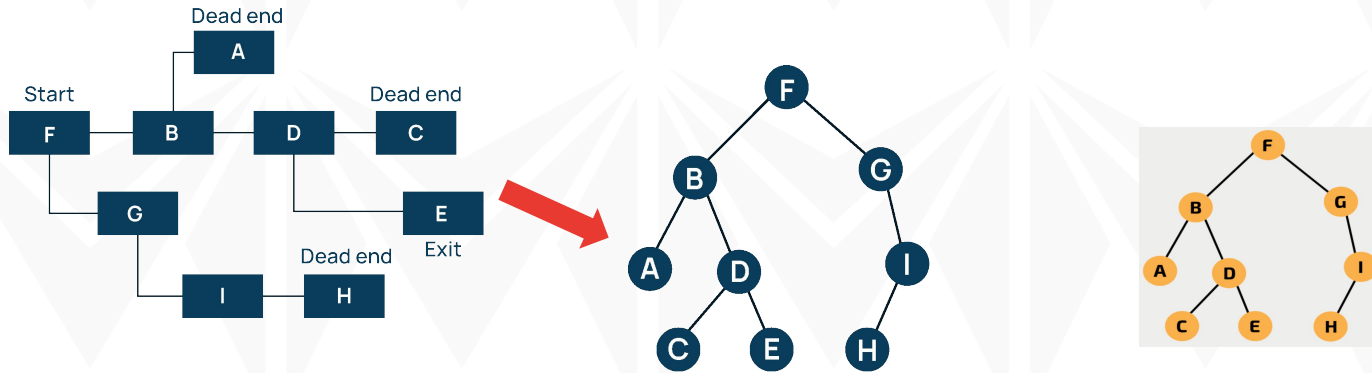
In every step, possible options are considered, and a decision is made about which option to follow.

When the algorithm reaches a final state, it checks whether that state is accepted or rejected and then “backtracks” to a previous state to try other unconsidered options.



BACKTRACKING

A maze can be represented by a binary search tree where backtracking traverses the tree and examine the leaves to check where there is an exit or a dead end in the maze.





DATA MINING

Deals with extracting valuable information from large data sets by unnoticed patterns and discovering underlying relationships within the data.

Helps with making informed decisions or predict future trends.

Techniques such as machine learning and statistical analysis are used for data mining.

Huge amounts of unconnected gathered data (i.e. Big data) can be interpreted with data mining using machine learning techniques.



DATA MINING

Examples of user activity data collected from a music-centric social network are:

- Rating songs, e.g. using a five-star song rating system.
- Sharing songs or playlists with other users.



DATA MINING

A company can mine the data and obtain answers to very important questions like;

- Which songs or artists are the most/least popular?
- Which features of the platform are used the most/least?



MID LESSON QUESTIONS

1. _____ deals with evaluating if a program works as expected or not before it is released, and using this knowledge to make it more effective.
 - a) **Performance testing**
 - b) Data mining
 - c) Pipelining
 - d) Backtracking



PERFORMANCE TESTING

Deals with evaluating if a system or a program works as expected or not before it is released, and using this knowledge to make it more effective.

A way to carry out this test is to;

- Come up with a set of test scenarios and success criteria.
- And measure the performance of the system or program against them.



PERFORMANCE TESTING

Examples of performance testing includes;

- Testing a program with erroneous data in order to check that it can handle errors effectively.
- Simulate real-life circumstances (i.e. stress testing) and check how well the system or program operates.



PIPELINING

Deals with dividing a task into a series of subtasks, where each subtask leads to the next.

Each subtask is executed by a different process in a series.

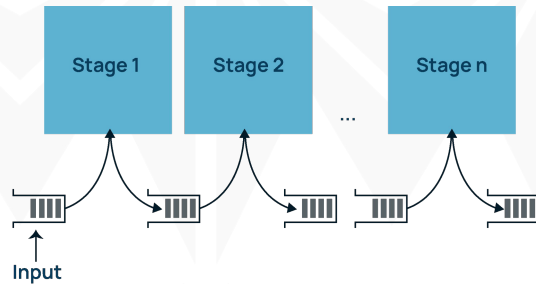
The result of a process feeds into the next one, and this is repeated until the whole task is completed.

In programming, an example of pipelining is when the results of a function is used as input to another function and this is repeated until the end result is produced.



PIPELINING

Consider an example of a program that calculates the total cost of an order by adding up the cost of the products and the VAT,



Stage 1	Stage 2	Stage 3
Retrieve the data from the database:	Calculate the cost of the products:	Calculate the total cost of the order:
Query the database to find the products in the order and the price per unit of each product.	Multiply the cost per unit by the number of units for each product. Add up all the intermediate costs.	Calculate the VAT on the cost of the products. Add up the total cost of the products and the VAT.



VISUALISATION

Deals with using techniques such as diagrams, graphs and images to illustrate what a problem entails, and how to approach its solution.

It works based on the knowledge that people often process and retain information better when it is presented in the form of an image instead of text.

Help humans (rather than computers) to solve problems more effectively.

Very useful to explain complex tasks, scenarios, or situations.



VISUALISATION

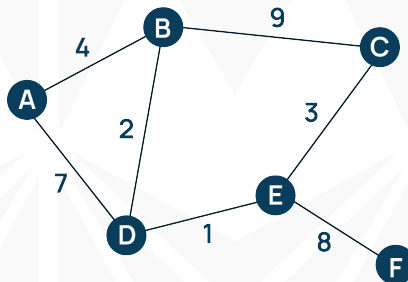
Using Dijkstra's algorithm to calculate the shortest path between two cities

Text

The distance from city A to city B is 4 km
The distance from city A to city D is 7 km
The distance from city B to city C is 9 km
The distance from city B to city D is 2 km
The distance from city C to city E is 3 km
The distance from city D to city E is 1 km
The distance from city E to city F is 8 km

Adjacency matrix

	A	B	C	D	E	F
A		4		7		
B	4		9	2		
C		9			3	
D	7	2			1	
E			3	1		8
F					8	





FURTHER READING RESOURCES

- Computational methods. Isaac Computer Science. (n.d.).
https://isaaccomputerscience.org/concepts/dsa_ctm_methods?examBoard=all&stage=all
- Hand, D. J. (2007). Principles of data mining. Drug safety, 30, 621-622.
- Salaria, R. S. (2018). Data Structures & Algorithms Using C++. KHANNA PUBLISHING HOUSE.



REFERENCES

1. *Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms. MIT press.*



DEPARTMENT OF
COMPUTER SCIENCE



THANK
YOU